

Notas Criptografia - Semestre VI

Abraham Nuñez

June 2, 2026

Contents

1	Introducción	3
2	Información	3
2.1	Prerrequisitos:	3
2.2	Evaluación:	3
2.3	Herramientas:	3
3	Temario	4
3.1	Descripción	4
3.2	Unidad 1	4
3.3	Unidad 2	4
3.4	Unidad 3	4
3.5	Unidad 4	4
4	Introducción a la criptografia	5
4.1	STRIDE	5
4.2	Origen de la criptografia	6
4.3	Criptografia	6
4.3.1	Herramientas	7
4.3.2	Cifrado Cesar	7
4.3.3	Tres protecciones principales de la criptografia	7
4.3.4	AES	8
4.3.5	YANAC	8
4.3.6	Codificar	8
4.3.7	Cifrar	8
4.3.8	Ofuscar	9
4.3.9	Hashear	9
4.3.10	Cifrado cesar	9

4.3.11	Aritmetica Modular	9
4.3.12	Binario	9
4.4	Hashing	9
4.4.1	Cifrado vs Hashing	10
4.4.2	DIGESTS	10
4.4.3	hashlib	11
4.4.4	Funciones de hash	11
4.4.5	Hashing	12
4.4.6	Seguridad en funciones de hash	14
4.5	Cifrado simetrico	14
4.5.1	AES	14
4.5.2	Seguridad en cifrado simetrico	16
4.6	Cifrado asimetrico	16
4.7	Integridad de mensajes, firmas y certificados	17
4.7.1	Message Authentication Code (MAC)	17
4.7.2	Cifrado y MAC	20
4.7.3	Firmas digitales	20
4.7.4	Curvas Elipticas	21
4.7.5	Transporte de Llaves RSA	21
4.7.6	Certificados	22
4.7.7	Ataques de repetición	24
4.8	Combinación de metodos simetricos con asimetricos	24
4.8.1	Agree Upon Transmission	24
4.8.2	Llaves de sesión	25
4.8.3	Diffie-Hellman DH	26
4.8.4	Protocolo Reto-Respuesta	27
4.8.5	Ramsonware	28
4.9	Cifrado simetrico avanzado	29
4.9.1	Cifrado autenticado con datos adicionales (AEAD)	29
4.9.2	Kerberos	30
4.10	TLS	30
4.10.1	Certificados X.509	31
4.10.2	Handshake	33
4.10.3	Verificación de certificados	37
4.10.4	Ataques a TLS	37
4.11	Examen	39

1 Introducción

- Profesor: Hector Xavier Limón Riaño
- Materia: Criptografía
- Contacto: xavier120@hotmail.com
- Carrera: Ciberseguridad e Infraestructura de Computo

2 Información

2.1 Prerrequisitos:

- Estructuras de datos (Python)
- Matematicas para la ciberseguridad
- Ciberseguridad
- Manipulacion de archivos de texto y binarios
- Manipulacion de cadenas de texto y binarios
- Manipulacion de datos binarios
- Sockets TCP

2.2 Evaluación:

- Exámenes: 60%
- Tareas y practicas: 20%
- Proyecto: 20%

2.3 Herramientas:

- Eminus
- Entorno controlado para exámenes
- Practical Cryptography in Python - Libro de Referencia

3 Temario

3.1 Descripción

Es un elemento fundamental de la protección de la información.

3.2 Unidad 1

- Cifrado Cesar
- Usos de la criptografia
- Criptografia y criptoanálisis

3.3 Unidad 2

- Funciones de hash
- AES
- Modos de operación de AES
- Padding
- Manejo de llaves y vectores de inicialización

3.4 Unidad 3

- Llaves publicas y privadas
- RSA
- PKCS

3.5 Unidad 4

- MAC, HMAC. CBC-MAC
- Firmas digitales
- Curvas

4 Introducción a la criptografía

- CIA: Confidencialidad -> Impide ver algo a quien no debe verlo. Integridad -> Evitar la manipulación de los datos. Disponibilidad -> Tener la posibilidad de acceder a los datos (que tengamos permiso) en cualquier momento.

Autenticar -> Probar que eres quien dices ser. Autorizar -> Dar acceso al que se autentica mediante permisos.

Secretos

Llaves privadas

Datos pueden estar en 3 estados:

- Reposo -> Esta almacenado en disco.
- Transito -> Estan viajando por la red.
- Uso -> Estan en la memoria

4.1 STRIDE

- STRIDE es un modelo de amenazas, utilizado para ayudar a razonar y encontrar amenazas a un sistema.
- Spoofing: Suplantar la identidad en redes informaticas haciendose pasar por una persona, entidad o sistema de confianza para engañar a la victima. Se puede mitigar por firmas digitales, contraseñas, etc. -> Autenticidad
- Tampering: Alterar, manipular o interferir con algo sin autorización, generalmente con el fin de causar daño, obtener beneficios ilegales o comprometer la integridad de un sistema, proceso o evidencia. Usa la tecnologia de hash para mitigarlo. -> Integridad
- Reputation: El acto de negar, por parte de una persona o sistema, haber realizado una acción, transacción o comunicación específica. -> No repudio
- Information disclosure: Acceso no autorizado o la revelación inadvertida de datos sensibles, confidenciales o privados a personas, sistemas o entidades que no deberían tener acceso a ellos. -> Confidencialidad
- Denegation of service: Hacer que un sistema, servicio o red sea inaccesible para sus usuarios legitimos. -> Disponibilidad

- Elevation of privileges: Otorgar permisos a usuarios que no deben de tener dichos permisos. -> Autorizacion

4.2 Origen de la criptografia

- Viene del griego kryptos que significa oculto y graphe que significa grafo o escritura.
- Es un ambito de la criptologia que se ocupa de las tecnicas de cifrado o codificado destinadas a alterar las representaciones linguisticas de ciertos mensajes con el fin de hacerlos ininteligibles.
- Se encarga del estudio de los algoritmos, protocolos criptograficos, y sistemas que se utilizan para proteger la información y dotar de seguridad a las comunicaciones y a las entidades que se comunican.
- El criptoanálisis es la parte que se dedica al estudio de los sistemas criptograficos con el fin de encontrar debilidades en los sistemas y romper su seguridad sin el conocimiento de información secreta.

4.3 Criptografia

- Esta en todos lados: ssh, criptomonedas y cualquier aplicación web.
- Es un pilar de la seguridad informática moderna.
- La criptografía no es solo cifrado, se utiliza también para autenticación e integridad de los datos.
- La criptografía bien implementada es efectiva sin importar:
 - Que tan bien educado este el adversario.
 - Que tantas computadoras tenga el adversario.
 - Si el adversario conoce los algoritmos usados.
 - Que tan motivado este el adversario.
- La efectividad de la criptografía depende usualmente del contexto.
- La buena seguridad depende de siempre de tus elecciones.
- Se debe de tomar la decisión adecuada de acuerdo al contexto del problema.

- Nunca utilices un metodo criptografico si no entiendes su contexto de uso y su correcta configuración.
- Nunca utilices un metodo criptografico usando la configuración por defecto a menos que la conozcas.
- Revisa bien los metodos criptograficos que aplicaras muchos tienen vulnerabilidades conocidas.
- TLS: Transport Layer Security, es la base para otras tecnologias como HTTPS y VPN.
- OpenSSL
- Determinismo -> Es predecible

4.3.1 Herramientas

- cryptography: Biblioteca de python con todos los algoritmos criptograficos que se usaran en el curso.
- gmpy2: Biblioteca python para el tratamiento matematico de numeros grandes.

4.3.2 Cifrado Cesar

- Consiste en reemplazar una letra de un mensaje por otra letra, o tambien, desplazar por un valor x (shift-cipher) el numero de letras.
 - Se puede aplica tanto a texto como a binario.
 - * Si es binario debe considerarse el modulo 256
 - * Si es texto debe considerarse el valor ASCII

4.3.3 Tres protecciones principales de la criptografia

- Autenticidad: Firmas digitales, Codigos MAC.
- Confidencialidad: Cifrado, intercambio de llaves.
- Integridad: Hashing.

4.3.4 AES

- Estandar de cifrado asimetrico.
- Advanced Encryption Standard.
- Es el algoritmo de cifrado simetrico mas usado en la actualidad.
- Lo utiliza TLS (HTTPS) y las funciones de cifrado de sistemas de archivos de un OS.
- Debe de saberse utilizar, tiene diferentes modos de operacion, algunos de ellos inseguros.
- Existen variantes, como AES-128 que utiliza una llave de 16 bytes.
- Una llave con contenido aleatorio es una llave valida.

4.3.5 YANAC

- YANAC: You Are Not A Cryptographer. Acronimo popular en la comunidad criptografica.
- Dont roll your own crypto.
- Es mas facil usar mal los metodos criptograficos que usarlos bien.
- Abstente de crear tus propios metodos criptograficos.
- Hasta expertos criptograficos han creado metodos con vulnerabilidades.
- Un nuevo metodo pasa por muchas revisiones de expertos antes de ser usado de forma practica.
- Incluso debes de abstenerte de hacer tus propias

4.3.6 Codificar

- Transformar de una representacion a otra sin perdida.
- Base64

4.3.7 Cifrar

- Es el proceso de convertir informacion sensible en un formato ilegible mediante un secreto (llave) a las que solo partes autorizadas tienen acceso.

4.3.8 Ofuscar

- Se refiere a encubrir el significado haciendola mas confusa y complicada de interpretar.

4.3.9 Hashear

- Es la funcion de aplicar una funcion de hash, es decir, que transforma un conjunto de datos en una cadena alfanumerica unica que actua como una huella digital del contenido original. No es invertible.

4.3.10 Cifrado cesar

- Se basaen reemplazar una letra de un mensaje por otra letra.
- Se puede aplicar tanto a texto como a binario.
- Si es binario debe considerarse el modulo 256, si es texto debe cuidarse el valor ASCII

4.3.11 Aritmetica Modular

- $4 \% 4$
- $(144 + 300) \% 256 = 188$
- $(188 - 300) \% 256 = 144$
- El resultado sera siempre un numero natural.

4.3.12 Binario

- El binario se manipula en paquetes de bytes (8 bits), no se puede manipular directamentes menos (al menos en las arquitecturas populares modernas).
- Tarea: Cifrado cesar para archivo de texto.

4.4 Hashing

- $3^{**}2 = 9 \rightarrow$ Biyectivas
- $22 \% 2 = 0 \rightarrow$

- El hashing es uno de los elementos principales de la seguridad criptografía. Involucra el concepto de una función de una sola vía y huella digital.
 - Una función hash solo funciona si cumple con los siguientes requisitos:
 1. Producen valores únicos y repetibles, es decir, son deterministas para cada entrada.
 2. El valor de salida no da ninguna pista (o patrón) con respecto a la entrada que lo produjo.
- Algunas funciones de hash son mejores que otras cumpliendo las propiedades antes mencionadas.
- Buenas: SHA-256, SHA-512
- No tan buenas: MD5, SHA-1. Estas no deberían ser usadas por sus vectores de seguridad.
- Solo funciona con cadenas binarias

4.4.1 Cifrado vs Hashing

- El cifrado permite tener confidencialidad en los mensajes, pero ayuda a verificar la integridad de los mismos.
- Confidencialidad: Solo las personas autorizadas pueden leer el mensaje.
- Integridad: Las personas no autorizadas no pueden cambiar el contenido del mensaje sin que dicho cambio pueda ser percibido.
- Solo porque un mensaje este cifrado (no pueda ser leído) no significa que no pueda ser alterado, incluso de formas que no tengan sentido después de cifrar.

4.4.2 DIGESTS

- Su traducción sería “resumen”
- Una función de hash produce un resumen o huella digital de algún contenido binario
- Este resumen puede utilizarse como medida para verificación de tampering (alteraciones de información), utilizadas en análisis forense.

- En una transmision la idea es transmitir el mensaje junto con su digest, para verificar que haya llegado sin alteraciones.

4.4.3 hashlib

```
import hashlib md5hasher = hashlib.md5(b'cadenaBin') md5hasher.hexdigest()
md5hasher.update(b'continuacionCadenaBin') md5hasher.hexdigest()
```

4.4.4 Funciones de hash

- Una funcion de hash toma un contenido de cualquier tamano(hasta vacio) y siempre produce un numero de un tamano fijo.
- Por ejemplo, MD5 produce una salida de 16 bytes
- Este tamano tiene una relacion con la seguridad de la funcion entre mas pequenos es peor.
- Dominio -> Codominio
- Una vez mas, las propiedades que una funcion de hash debe de cumplir son:
 - El mismo documento siempre produce el mismo digest.
 - El digest parece aleatorio, si tienes el digest eso no te da pistas del documento de entrada.
 - Idealmente cada documento con contenido diferente deberia tener un digest diferente (lo cual en realidad no es posible).
- Los digests son como huellas digitales: si tienes al sujeto se puede verificar que sea quien dice ser mediante su huella digital.
- Sin embargo, si solo tiene la huella digital no es facil determinar el sujeto que la posee.
- Es facil calcular el digests de un documento, pero si tiene solo el digest es muy dificil determinar el documento que lo produjo, entre mas dificil mejor.
- Por esta razon son llamadas funciones de una sola via, se puede ir de entrada a la salida pero no al revés.

4.4.5 Hashing

- En computacion, el termino funcion de hash casi siempre se refiere a una funcion criptografica.
- Sin embargo, el termino es general
- Por ejemplo: Una funcion hash que establece si un numero es par o impar. Esencialmente, las funciones hash relaciona elementos de un dominio muy grande, con elementos de un codominio relativamente pequeno.
- El tamano del codominio depende de la funcion de hash, por ejemplo, ya se dijo que MD5 genera digest de 16 bytes. Cual es el tamano del codominio para MD5?
- SI se considera un numero muy grande de documentos de entrada es posible que muchos de ellos produzcan el mismo hash.
- Dado lo anterior, las funciones de hash tienen perdida. Se pierde informacion de la fuente al digest.
- Que tenga perdida es importante, de lo contrario habria una forma de regresar del digest al documento original.
- Considerando de nuevo la funcion par
- Cualquier numero puede dar como resultado 0 (par) o 1 (impar)
- Seria posible saber, dado un valor 0 o 1 que numero produjo dicho resultado
- Esta funcion tiene todas las propiedades: tienen perdida, comprime la entrada y es consistente ya que dada la misma entrada siempre produce la misma salida.
- Las propiedades de consistencia, compresion y perdida son comunes en cualquier funcion de hash pero no suficientes para ser seguras o criptograficas.
- Se requieren las sig. propiedades:
 - Resistencia de preimagen.
 - Segunda resistencia de preimagen.

– Resistencia de colisiones.

1. Resistencia de preimagen

- Una preimagen es el conjunto de entradas de una funcion de hash que producen, en el caso de par o impar, una preimagen son todos los numeros pares que den valor 0.
- Una preimagen de una funcion de hash H y un valor de hash k es el conjunto de valores x por el cual $H(x) = k$
- Al tener un digest, existe un numero infinito de documentos que produjeron dicho digest.
- Eso no quiere decir que dado un digest sea facil encontrar un elemento de la preimagen
- La resistencia de preimagen es basicamente: si tengo un digest y no se como se obtuvo, no pudo ni siquiera encontrar un elemento en la preimagen sin realizar una cantidad ridicula de trabajo computacional.
- Intratable -> Se puede resolver, pero cuesta demasiado
- No computable -> No se puede resolver con una computadora
- Idealmente, los elementos de la preimagen deberian estar muy separados entre ellos y no tener un espaciado predecible
- El hecho de que las funciones de hash tengan perdida ayuda a la resistencia de preimagen
- Si una funcion de hash tiene buena resistencia de preimagen, los ataques en este sentido seran aleatorios o bien consistiran en enumeraciones muy largas (fuerza bruta) hasta encontrar una coincidencia, lo cual no es computacionalmente tratable.
- Al proceso de intentar encontrar un elemento de la preimagen dado un valor de hash se le llama invertir el hash
- Encontrar un inverso deberia ser muy dificil.

2. Segunda resistencia de preimagen

- Si tu tienes al menos un documento que produce un hash conocido, sigue siendo muy dificil encontrar otro documento que produzca el mismo hash.
- Esto funciona porque no existe patron repetible en los elementos de la preimagen.

3. Resistencia a colisiones

- Significa que es difícil encontrar dos entradas que produzcan el mismo hash y garantiza que sea computacionalmente inviable encontrar dos entradas diferentes que produzcan el mismo valor hash.
- No un hash específico sino cualquier hash.
- Es más fácil encontrar una colisión en un hash cualquiera que encontrar un elemento de la preimagen de un hash dado.
- Propiedad avalancha: El valor de hash cambia drásticamente y de forma impredecible, un mínimo del 50% de los bits del hash debería ser diferente.

4.4.6 Seguridad en funciones de hash

- Hash es usado para el almacenamiento de passwords, pero siguen teniendo problemas de seguridad, el más común es la situación donde dos usuarios usen la misma contraseña y por tanto, generen el mismo hash, existen muchas soluciones (A2F, Token físico, etc) pero la más recomendada es agregar un salt.
 - Salt: Es un valor usualmente aleatorio que se debe de agregar al password antes de aplicarse a la función de hash. Debido a la propiedad de avalancha, con el salt el hash resultante cambiaría drásticamente. Y como son aleatorios, es poco probable que dos usuarios con la misma password tengan el mismo salt.

```
salt = os.urandom(16)
```

4.5 Cifrado simétrico

- Cifrado que usa la misma llave para cifrar y descifrar, la llave es un secreto compartido.

4.5.1 AES

- Advanced Encryption Standard
- Lo utiliza TLS (HTTPS) y funciones de cifrado de sistemas de archivos de un OS.

- AES es un cifrador de bloque, cifra por bloques de 16 bytes, si no cumple con los 16 bytes se debe de agregar un padding (datos de relleno).
- Tambien puede cifrar un byte a la vez, esto se conoce como cifrado de flujo y no necesitan padding.
- En general, es mas eficiente cifrar bloques que en flujo.

1. Modos de operacion de AES

(a) ECB - Electronic Code Book

- Cualquier texto plano de 16 bytes tiene 16 bytes correspondientes de salida.
- Cada bloque es tratado independientemente, esa es su debilidad.
 - Puede generar un ataque de texto plano escogido, que se basa en tener muchos mensajes de estructura parecida para enganar con un mensaje falso con una respuesta predecible.
- Parametros: Texto plano y llave.

(b) CBC - Cipher Block Chaining

- Despues de cifrar el bloque aplicar una funcion XOR con el siguiente bloque en texto plano, el resultado se cifra y se guarda ese bloque. Para descifrar se tiene que invertir el proceso.
- Para el primer bloque a cifrar, se utiliza el IV dado que no hay un bloque anterior para hacer XOR, el IV debe de ser del mismo tamaño del bloque (16 bytes).
 - Parametros: Texto plano, llave e IV
 - Debe de tener su respectivo padding.

(c) CTR - Counter Mode

- No requiere de padding
- Genera un key stream de cualquier longitud a partir de una llave AES de al menos 128 bits (16 bytes), luego aplica XOR para combinarlos.
- CTR incrementa el contador en uno y asi sucesivamente. Si el texto plano no es multiplo de 16, los ultimos 16 bytes del key stream se truncan para corresponder al texto.

- Al IV se le llama nonce.
- CTR puede cifrar de forma independiente.
- Parametros: key, nonce, texto plano.

4.5.2 Seguridad en cifrado simetrico

- Para tener un cifrado seguro y efectivo se necesita:
 1. Cifrar el mismo mensaje de forma diferente cada vez que se haga, aunque se use la misma llave.
 2. Eliminar patrones predecibles entre los bloques. (Maleabilidad entre bloques)
- Para resolver el primer problema, se utiliza un vector de inicializacion (IV), el cual es una cadena aleatoria externa al texto plano y la llave. El IV es publico ya que solo hace que los mensajes no sean repetibles y no generar patrones.
- Nunca debe reusarse el mismo IV
- Existen ataques donde si conoces el texto plano y el IV (suponiendo que lo reutilizan) puedes obtener el key-stream.

4.6 Cifrado asimetrico

- Las tecnicas asimetricas son variadas, no se limitan solo a cifrado
 - Por ejemplo: firmas digitales e intercambio de llaves
 - Algoritmo RSA
 - Existen algoritmos derivados de las curvas elipticas
1. Dos llaves
 - Se necesitan dos llaves: una publica y otra privada
 - Un mensaje se cifra con la llave publica
 - Ese mensaje solo se puede descifrar con la llave privada
 - La llave publica se comparte y la privada no.
 2. RSA
 - RSA esta casi obsoleto, debido a multiples vulnerabilidades

- Sirve para ejemplificar el cifrado asimétrico
- Factorización de primos
- Las llaves son básicamente números primos muy grandes que se
- Estos números son coprimos
- Si solo se tiene uno de los dos

4.7 Integridad de mensajes, firmas y certificados

- MACS
- Firmas digitales
- Certificados
- “Keyed Hashes”
- Uso de criptografía asimétrica para integridad y autenticidad de mensajes
- Certificados y su diferencia con las llaves

4.7.1 Message Authentication Code (MAC)

- Un cifrado puede ser alterado de formas que tenga sentido.
- Un MAC es cualquier código o dato transmitido junto con el mensaje que puede ser evaluado para determinar si el mensaje fue alterado.
- Usualmente se utiliza un hash como
- Un MAC seguro no solo es código (hash) también debe incluir una llave.
- Una llave MAC no está relacionada con el cifrado
- Sirve para asegurar que el código de autenticación de mensaje solo puede ser computado por las entidades que conocen la llave.
- No se debe asumir que el atacante no conoce la función de hash, recuerda que una buena seguridad significa. . .
- MAC protege la integridad del mensaje

- Un atacante sin la llave no debe poder alterar el mensaje sin que se pueda detectar.
- Mientras la llave se mantenga privada, el MAC

1. HMAC

- Hash-based Message Authentication Code
- Es basicamente un hash keyed
- Las funciones de hash son deterministas, dada la misma entrada siempre se produce la misma salida.
- Solo agregar al inicio o al final
- $H(K \text{ XOR opad} + H((K \text{ XOR ipad}) + \text{text}))$
 - H es la funcion de hash
 - K es la llave que debe de cumplir ciertos requisitos
 - text es cualquier binario que representa el contenido de entrada
 - opad e ipad son dos valoresi
 - B es el tamano del bloque de H (128 bytes)
 - (a) Si K es mas corto que B, entonces a K se le tienen que agregar ceros de padding hasta ser de longitud B.
 - (b) Si K es mas largo que B, entonces K se reduce aplicando H a K, osea $H(K)$
 - (c) Si se aplico 2 y la llave quedo muy corta, entonces se aplica 1
 - Si se aplica lo anterior, siempre se va a tener una llave de longitud B.
 - ipad se define com el byte 0x36 repetido B veces
 - opad es el byte 0x5c repetido B veces
 - Los pads parecen arbitrarios pero tienen un sentido de seguridad matematico que ayuda a mitigar
- Primero computas

2. CBC-MAC

- Similar a AES-CBC
- Recordar que CBC es Cipher Block Chaining
- Un MAC a menudo se le llama “tag” (mensaje) del mensaje

- Es info extra que se asocia al principio del contenido
- En notacion matematica un tag se denota a menudo como t
- Asi, un MAC sobre un mensaje m_1 produce un tag t_1
- El par (m_1, t_1) es transmitido al receptor para ser verificado
- AES solo puede cifrar 128 bits de bloque a la vez.
- AES-CBC encadena la salida de un bloque cifrado con el siguiente bloque.
- Si se altera uno se alteran todos los bloques siguientes.
- El ultimo bloque de CBC es en si un MAC.

(a) MAC Prepend Attack

- Es posible mandar un mensaje manipulado y hacer que parezca que es el mensaje M_1 o M_2 , suponiendo que esten generados por la misma llave.
- M_3 es una mezcla de M_1 y M_2 .
- No es necesario conocer la llave de MAC.

i. Proceso de ataque, anteponiendo M_1 a M_2

- Recuperar el primer bloque de M_2 , denominado M_{21} (como es AES se toman los primeros 16 bytes)
- Recuperar el resto de bloques de M_2 , denominado M_{2r}
- Obtener $M'_2 = t_1 \text{ XOR } M_{21}$
- $M_3 = M_1 + M'_2 + M_{2r}$ (+ significa concatenacion)
- Con esto $t_3 = t_2$
- Basicamente lo que hace el atacante es reemplazar el primer bloque de M_2 por uno manipulado.
- La victima vera todo el mensaje M_1 completo (con su padding), un bloque que se vera posiblemente corrupto y el mensaje M_2 sin su primer bloque.

A. Soluciones

- Agregar siempre la longitud del mensaje al inicio. Esto ayudara a corroborar si hay información extra.
- Si se cifraron los datos usando AES-CBC, no se debe usar la misma llave para cifrar ni para obtener el MAC.
- El IV debe ser diferente a un valor de cero.

4.7.2 Cifrado y MAC

- Encrypt-Then-MAC: El MAC se aplica al texto cifrado.
- MAC-Then-Encrypt: Se aplica al texto plano, luego se cifra el texto plano junto al MAC.
- Encrypt-And-Mac: Se aplica MAC sobre el texto plano, pero no se cifra el MAC, solo el texto plano.
- Recomendación: Siempre usar Encrypt-Then-MAC.
- El ataque de padding oraculo a CBC funciona en MAC-Then-Encrypt.

4.7.3 Firmas digitales

1. RSA

- Dado que los MAC usan llaves simetricas, si se compromete la llave puede ambos lados estan en riesgo de spoofing (usurpación de identidad).
- Las firmas intentan mitigarlo mediante criptografia asimetrica. Ya que nos existen llaves “compartidas”.
- La llave publica solo se puede descifrar con la llave privada y vise-versa.
- Si se cifra con la llave privada, puede ser descifrado por la llave publica.
- Tambien pueden usarse para firmar listas de revocación de certificados CRL's.
- Autenticidad en reversa: El receptor no sabra de quien es el mensaje pero el emisor sabe que solo podra ser procesada por la entidad autorizada.
- Existen muchos problemas con el cifrado asimetrico (RSA), es ineficiente para archivos grandes, no viene comprobación de integridad y solo puede procesar archivos del tamaño de modulo (2048 bits).
- Para solucionar eso, se hace lo siguiente: Cifrar el hash del mensaje, dando tanto integridad como autenticidad.
 - (a) Calcular el hash del mensaje.

- (b) Cifrar el hash mediante la llave privada, este hash cifrado es la firma del emisor.
- (c) El receptor calcula el hash del mensaje, descifra la firma mediante la llave publica y comprueba que coincidan ambos hashes.
- La firma debe de aplicarse siempre al texto plano, nunca al cifrado.
- El cifrado con la llave publica es confidencialidad.
- El cifrado con la llave privada es autenticidad.
- El padding para firmas RSA es PSS.
- Es comun usar tecnicas asimetricas al inicio de la comunicaci3n para el establecimiento de confianza e intercambio de llaves simetricas. Despues todo el proceso se hace con llaves simetricas.

4.7.4 Curvas Elipticas

- Alternativa a RSA
- Se usa para firmas digitales y el intercambio de llaves.
- Se usa ECDSA para firmar.
- Parametros:
 - Tipo de curva (P-384 - SECP384r1)

4.7.5 Transporte de Llaves RSA

1. Alguna de las partes genera la llave simetrica. (probablemente el servidor)
2. Se cifra la llave simetrica con una llave publica.
3. La otra parte tiene una llave privada.
4. Se transmite la llave cifrada y se descifra.
5. Seguridad hacia delante (Forward Secrecy)

4.7.6 Certificados

- Prueban la pertenencia de las llaves publicas.
- Modelo PKI (Public Key Infrastructure)
 - Un registro global donde se almacenaban las llaves publicas asociadas a una entidad.
 - La llave publica se compartia.
 - No era escalable.
- Un certificado solo son datos, incluyen:
 - Una llave publica
 - Metadatos del propietario de la llave (identidad del propietario de la llave, identidad del issuer, fecha de expiración, numero de serie, etc)
 - * La identidad del propietario puede ser un nombre, email, URL, etc.
 - Firma digital sobre todo el contenido del certificado
 - Firma por la entida de confianza (issuer)
- Se comparten los certificados en lugar de las llaves publicas.
- Una entidad quiere un certificado:
 1. Genera su par de llaves
 2. La entidad de confianza recibe la llave publica y genera el certificado
 3. La entidad de confianza firma el certificado con su llave privada
- Para evitar que un agente malicioso use un certificado que no es suyo, se usa el proceso reto-respuesta (challenge-response).
 - Se basa en pedirle que firme un mensaje con la llave privada y verificarla con la llave publica del certificado. Si coincide, entonces si puedes confiar.
- Protocolo reto-respuesta
 1. El cliente recibe la llave publica del servidor (certificado).
 2. El cliente le manda un reto al servidor.

3. El cliente le pide al servidor que firme el dato.
 4. El servidor firma el dato y regresa la firma.
 5. El cliente comprueba la firma con la llave publica.
- Quien firma es quien quiere ser autenticado
 - Siempre se firma con la llave privada
 - Se verifica la firma con la llave publica
 - La firma tiene dos propiedades de seguridad
 - Integridad
 - Autenticidad
 - Al issuer se le llama Certificate Authority (CA)
 - Estos tambien tienen sus propios certificados con nombre de identidad y su llave publica.
 - Certificados:
 - Metadatos del propietario (subject)
 - Metadatos de la propiedad de confianza (CA/issuer)
 - Llave publica
 - Firma del CA
 - El certificado no tiene nada que ver con el cifrado, ni con la protección del canal en general.
 - El certificado no se usa directamente para autenticar, eso se hace con firmas digitales.
 - El certificado solo es un mecanismo para confiar en la llave publica
 - Criptografia asimetrica
 - Firmas digitales
 - Intercambios de llaves
1. Revocación de certificados
 - Si la llave privada de un certificado creado por un CA es comprometido, el CA debe hacer saber a los demas que ya no es confiable.

- Dado que no se puede tener un sistema offline de revocación, existen dos alternativas:
 - Tener un componente en tiempo real
 - No se puede actualizar en tiempo real
- Un ejemplo es OCSP (Online Certificate Status Protocol)
 - Revisa el estatus del certificado en tiempo real
 - Tambien revisa CRL's (listas de certificados revocados)
- La recomendación final siempre sera proteger las llaves privadas, de preferencia en un lugar offline y/o con permisos minimos con un cifrado adicional.
- Los certificados deben tener un tiempo de vida relativamente corto.

4.7.7 Ataques de repetición

- Un mensaje legitimo es utilizado por un atacante tiempo despues cuando el mensaje ya no es valido.
- Soluciones:
 - Se agrega un nonce (numero aleatorio) para hacer seguimiento de mensajes recibidos.
 - Usar timestamp (fecha del mensaje) para rechazar mensajes demasiados viejos. Aunque es necesario una ventana de tiempo para lidiar con retrasos.
 - Combinar timestamp y nonces aleatorios, rechazar mensajes muy viejos ademas que tengan el nonce repetidos dentro de la ventana de tiempo permitida.

4.8 Combinación de metodos simetricos con asimetricos

- Intercambio de llaves (simetricas)
- RSA cifra llaves AES compartidas en la comunicación

4.8.1 Agree Upon Transmission

- Transmite un solo flujo de bytes con todos los datos necesarios concatenados

- Sirve para transferir un solo mensaje
- Tienes: $llaves_{aes}$, $llave_{mac}$, iv cifradas con la llave publica
- Llave publica del receptor (viene con el certificado)
- Cifras las llaves con la llave publica (RSA)
- Firmar llaves planas
- Cifras el mensaje con la $llave_{aes}$ e IV
- Calcular el HMAC sobre todos los datos
- De sesiones cortas
- Es practico para mensajes largos
- Llave AES, IV y llave MAC cifradas con la llave publica del receptor
- La firma digital sobre la concatenación (no cifrada) de la llave AES, IV y llave MAC.
- Los bytes del mensaje transmitido, cifrados con la llave AES e IV .
- El HMAC sobre la transmisión utilizando la llave MAC.
- Descifra mediante su llave privada, obtiene la llave AES, IV y MAC
- Verifica la firma digital, mediante el certificado del emisor.
- Calcula HMAC de 1, 2 y 3 con la llave MAC y lo compara con 4.
- Descifra 3 mediante la llave AES y el IV .

4.8.2 Llaves de sesión

- Se usa una sola vez y se descarta permanentemente, es decir, son efimeras.
- Llaves para el cifrado en masa
- Cifrado en masa: Se cifra una gran cantidad de datos.
- Payload: cargo util
- Son faciles de crear
- Se pueden derivar de un secreto base usando funciones de derivación de llaves.

4.8.3 Diffie-Hellman DH

- Criptografía asimétrica
- Una variante es con curvas elípticas ECDH
- Intercambia parámetros públicos para computar simultáneamente la misma llave en ambos lados, esto se llama Key Agreement
- Primero crea un par de números matemáticos, uno privado y público. Esto para cada participante.
- Ambas partes intercambian sus números públicos.
- Con el número público externo y el número privado local se genera la misma llave compartida en ambos lados.
- El resultado es una llave simétrica o un dato que permite derivar una llave simétrica.
- Requiere comunicación bidireccional.
- Los parámetros de DH deben ser iguales
 - Tamaño de la llave (2048)
 - Generador (2, 5)
- Se recomienda usar HKDF para derivar la llave
- DH no tiene autenticación integrada
- Una o ambas partes deben de firmar públicos intercambiados.
- Son preferibles para el transporte de llaves antes que RSA.
- Principalmente porque ofrecen “Confidencialidad hacia delante” (Forward Secrecy) dada la naturaleza de las llaves efímeras, si una llave es comprometida, no se revela nada acerca de comunicaciones previas.
- A veces para establecer un canal de confianza necesitas transmitir datos con poca protección (pueden ser manipulados)
- Eventualmente se llega a tener un canal que se cree protegido (ya se hizo bien el intercambio de llaves)

- Mientras tanto se fueron calculando hashes de todos lo que se transmitio

1. Autenticación con DH y ECDH

- El intercambio de llaves requiere una llave publica RSA o ECDSA, con su certificado firmado. Se usan para establecer identidad de cada parte, firmando info efimera durante el proceso de generación.
- Ademas se agrega challenge-response.

4.8.4 Protocolo Reto-Respuesta

- “Yo te envío un reto aleatorio. Si puedes responder correctamente, demuestras que posees la llave privada.”
- Paso 1: Alice envía su certificado
 - Alice $\rightarrow$ Bob
 - El certificado contiene:
 - * Nombre Alice
 - * Llave pública Alice
 - * Firma del CA
 - Bob verifica:
 - * Certificado válido
 - * No expirado
 - * Firmado por una CA confiable
- Paso 2: Bob genera un reto
 - Bob crea un número aleatorio:
 - Reto (Nonce) = 93741852 (Porque debe ser imposible predecirlo.)
- Paso 3: Bob envía el reto
 - Bob $\rightarrow$ Alice (93741852)
- Paso 4: Alice firma el reto
 - Alice calcula: Firma = Sign(93741852) usando la llave privada de Alice.

- Con RSA: $\text{Firma} = \text{Reto}^d \bmod n$
- Paso 5: Alice responde
 - $\text{Alice} \rightarrow \text{Bob} (\text{Firma})$
- Paso 6: Bob verifica
 - Bob utiliza: Llave pública de Alice obtenida del certificado.
 - Verifica: $\text{Verify}(\text{Firma}, \text{Reto})$
 - * Si la verificación es correcta: Sólo alguien con la llave privada pudo generar esa firma.

4.8.5 Ramsonware

- Software que cifra archivos y se rehusa a liberarlos hasta que le pagues al extorsionador que esta detras de todo.
- El atacante genera dos llaves permanentes.
- El atacante hardcodea la llave publica permanente en el ramsonware
- Cuando el malware se activa se crean un par de llaves locales
- Se cifra la llave privada local con la llave publica permanente y se elimina la versión plana.
- El malware cifra los archivos usando AES, cada archivo cifrado tiene una llave diferente que es cifrada con la llave publica local, solo pueden ser descifradas por la llave privada local.
- Cuando se haga el pago, el malware envia la llave privada local cifrada al servidor CC
- Con la llave privada permanente se descifra la llave privada local y se reenvia a la victima
- Ahora se pueden descifrar las llaves AES.
- La llave privada permanente nunca es revelada.

4.9 Cifrado simetrico avanzado

4.9.1 Cifrado autenticado con datos adicionales (AEAD)

- Authenticated Encryption with Aditonal Data
- Proveen autenticidad, integridad y confidencialidad.
- Reemplazan a los MAC's
- Es mejor usar AEAD antes que los MAC

1. AES-GCM (Galois/Counter Mode)

- Permite cifrar y autenticar con una sola llave
- IV de tamano de 12 bytes
- No usa padding
- Recomendable usar junto a Scrypt o HDKF
- Datos adicionales no se cifran pero deben de autenticarse mediante la llave derivada.
- Al terminar el proceso se obtiene un tag.
- No usar IV 0
- Segmentar lo que transmites
- Un IV difeente para cada segmento
- No transmitir mas de 64 gb con la misma llave
- No usar el mismo par de llave e IV (nonce)
- GCM usa IV's de 12 bytes, a los 4 bytes restantes se les agrega un padding de ceros, para tener un contador de 16 bytes.

2. AES-CCM

- Basado en CBC
- Tag computado similar a CBC-MAC
- IV de entre 7 a 13 bytes
- Puede cifrar datos mayores a 64 GB
- No es tan eficiente como GCM

3. Chacha20-Poly1305

- Combina un cifrador de flujo (ChaCha20) con un algoritmo de MAC (Poly1305)
- No se han encontrado vulnerabilidades en este algoritmo
- Suele ser mas rapido que AES
- Es verdaderamente de flujo
- Usa un IV de 12 bytes.

4.9.2 Kerberos

- Es un protocolo muy conocido basado en cifrado simetrico para autenticación.
- Es un intermediario entre dos entidades que se desean comunicar siendo un tercero de confianza.
- Si el cliente se autentica primero al servicio de Kerberos, le permite el acceso a los servicios que esten conectados a Kerberos
- Componentes basicos:
 - Servicio de autenticación
 - Servidor de tickets

4.10 TLS

- Es una combinación de protocolos que trabajan en conjunto y soporta un numero de combinaciones de algoritmos y protocolos.
- HTTPS basicamente encapsula trafico HTTP dentro de TLS.
- TLS permite a un cliente verificar la identidad de un servidor y establecer una sesion segura y mantener llaves compartidas de cifrado.
- Las llaves se mantienen secretas incluso si un atacante intercepta el trafico.
- Usa PKI
- Si una entidad quiere TLS requiere de un certificado X.509
- Brinda seguridad a TCP/IP.
- Existen dos versiones 1.2 y 1.3.

4.10.1 Certificados X.509

- Su proposito es ligar la identidad de un subject a una llave publica bajo la firma de un issuer.
- Tiene 3 posibles campos, pueden tener varios niveles de sub-campos.
 - Certificate
 - * (a) Version Number
 - * (b) Serial Number
 - * (c) Signature Algorithm ID
 - * (d) Issuer Name (describe la identidad pretendida)
 - * (e) Validity Period
 - i. Not Before
 - ii. Not After
 - * (f) Subject Name (describe la identidad pretendida)
 - * (g) Subject Public Key Info
 - i. Public Key Algorithm
 - ii. Subject Public Key
 - * (h) Issuer Unique Identifier (optional)
 - * (i) Subject Unique Identifier (optional)
 - * (j) Extensions (optional)
 - Certificate Signature Algorithm
 - Certificate Signature
- La version 3: Incluye el campo adicional “Subject Alternative Name”
- Una vez identificados el issuer y el subject, los campos restantes son la llave pública del subject y la firma del contenido del certificado.
- Los certificados vienen firmados por un issuer.
- Si quiero comprobar un certificado tambien necesito el certificado del issuer (la llave publica del issuer).
- Se necesita una cadena de certificados, hasta llegar al certificado raiz (autofirmado).
- Si quieres engañar a alguien para entrar a un sitio web falso que necesitas?

- Sitio web con dominio casi igual.
 - * IDS/IPS
 - * Educar al usuario
 - Si alguien quiere tener un certificado, necesita un Certificate Signing Request (CSR) y enviárselo al CA. Esta petición tiene todos los campos que un certificado X.509 solo que sin los campos del issuer.
 - El CA usa su propio certificado y llave privada para generar el certificado con los campos faltantes. El issuer del certificado debe ser igual al campo de subject del certificado de quien firma.
 - Cuando los campos son rellenados el CA firma el certificado con su llave privada.
 - El CSR solo tiene la llave pública.
 - Los certificados están en formato DER o PEM.
 - Prueba de posesión: El CSR está firmado por la llave privada del subject.
 - En un certificado auto-firmado el subject e issuer son iguales.
 - Generar una llave privada (2048 bits) y el CSR (durante su creación se extrae la llave pública y se agrega a la petición).
 - El issuer genera el certificado firmandolo con su llave
 - Con este certificado se pueden firmar nuevos certificados.
 - Para los nuevos certificados se deben de crear sus respectivas llaves.
 - Con los certificados del CA, se pueden generar certificados asociados a los nuevos CSR.
 - Se pueden agregar limitaciones usando un archivo v3.ext
 - El CA firma el CSR del subject, el issuer del nuevo certificado debe ser igual al subject del certificado del CA.
1. Requerimientos
 - Se debe probar tener el control sobre el dominio del sitio.

- Mediante protocolos como ACME o webhosting.
- Envenenamiento de DNS
 - Solo funciona si usas http o si se tiene una llave privada asociada a la cadena de certificados.
 - * Se mitiga para http con redirección hacia https
 - Por que no funciona con https?
 - * Por la comprobación de certificados
- Yo tengo un certificado
- Como lo avala un tercero?
- Una cosa es que se compruebe que la entidad es la propietaria verdadera del certificado
- Y otra es que el certificado sea confiable.

4.10.2 Handshake

- Tiene como objetivos: establecer identidad y derivar llaves de sesión para el transporte seguro.

1. TLS - 1.2

- (a) Comienza con un cliente enviando un mensaje hello al servidor. Este mensaje contiene info de configuración, tipo nonce. También esta la lista de Cipher Suites.
 - El objetivo es ponerse de acuerdo en cuanto al algoritmo y protocolos criptográficos a usar. La Cipher Suite da opciones.
 - La seguridad depende del Cipher Suite usado.
 - El cipher suite incluye:
 - Intercambio de llaves
 - Firma digital
 - Cifrado en masa
 - Hashing
- (b) El servidor recibe un hello cliente, escoge uno de los cipher suites propuestos del cliente. Cuando escoge uno, envía una respuesta con la siguiente info:
 - Hello con nonce aleatorio del servidor
 - Certificado del servidor

- Parte publica del intercambio de llaves DHE O ECDHE, en caso de usarse Diffie-Hellman y no RSA.
- (c) El cliente hace el proceso como autentificar al servidor, derivar la llave y empezar a enviar datos cifrados.
- TLS solo autentica el servidor, el cliente no envia un certificado para autenticarse.
 - Cuando ambas partes se autentican con un certificado se le llama Mutual TLS o MTLS.
 - Para autentificar al cliente se usa mas la identidad del usuario como user-password entre otros. Esto despues de establecer una sesión segura con una llave compartida.
- (d) El handshake no genera una llave en si, sino un PMS (pre-master-secret) junto a otros datos (nonce de ambas partes) para obtener el master secret.
- Este master secret se usa para generar las llaves de sesión para la comunicación en masa.
 - En el caso de usar RSA, no es necesario usar firmas dado que se envian las llaves publicas (en los certificados).
 - Mientras que DHE y ECDHE se derivan de manera simultanea y efimera. Ademas que debe de ser autentificado, al menos del lado del servidor, usando la llave privada del servidor de larga duración (RSA o ECDSA). El cliente cuando recibe los datos firmados usa la llave publica en el certificado del servidor para verificar la firma.
 - Es recomendable usar DH/ECHD siempre.
- (e) Finalizado el intercambio de llaves, toda la info debera ir cifrada y autentificada.
- (f) El cliente envia un mensaje llamado “ChangeCipherSpec” que indica que todos los mensajes siguientes usaran el CipherSuite negociado.
- (g) El servidor contesta con su propio CHangeCipherSpec.
- (h) Finalmente, cada parte manda un mensaje ya cifrado “Finished” que completa el handshake. Este finished contiene todos los hashes del handshake enviado. Asegurando que ningun atacante altero alguna parte del handshake.
- En caso que no pase la comprobación, se cierra el canal dado que esta comprometido.

- (i) TLS segmenta las transmisiones en paquetes, cada paquete está cifrado y autenticado, además, están estructurados en un formato llamado `TLS_CIPHERTEXT`.
- (j) Son de 16 kilobytes y son similares al struct de C.
- (a) Derivación de llaves
 - El PMS se expande al master secret usando una función pseudo-aleatoria (PRF), que toma un secreto y lo expande a más bytes, usando un mecanismo HMAC-SHA256 para expandir bytes de entrada a un tamaño arbitrario.
 - El PMS se expande hasta 48 bytes para generar el master secret.
 - El master secret también se expande (usando PRF) a tantos bytes sea necesario para obtener las llaves de sesión e IV's requeridos.
 - Del master secret se obtiene el `key_block` que varía del tamaño dependiendo del cipher suite.
 - El `key_block` tiene máximo 6 parámetros, esto depende del Cipher Suite:
 - Cliente MAC key
 - Servidor MAC key
 - Cliente cifrado key
 - Servidor cifrado key
 - Cliente IV
 - Servidor IV

2. TLS - 1.3

- Desecha la mayoría de cipher suites, quedando solo 5 que están basados en AEAD.
- RSA ya no se usa para transporte de llaves.
- Solo DHE y ECDHE.
- El handshake se hace en una sola ronda de mensajes.
- Cliente envía su Hello junto a Cipher Suites y su llave pública (ECDH).
- El servidor recibe el Hello del cliente, envía al cliente su Hello con la elección del Cipher Suite, su llave pública, el certificado del servidor, la firma del certificado y el finished.

- El cliente recibe todo lo anterior del servidor y envia su finished.
 - TLS 1.3 elimina la latencia de transmisión.
- (a) Datos miscelaneos
- Una sesión puede tener varias llaves simetricas.
 - Confidencialidad (transporte en si)
 - Llave emisor-receptor
 - Llave receptor-emisor
 - Autenticación/Integridad (MAC)
 - Llave emisor-receptor
 - Llave receptor-emisor
 - Con DH o transporte RSA generas un PMS, que permite generar un master secret.
 - Con el PMS puedes tener un generador de llaves
 - Intercambiar hellos (nonces, certificados, lista de cipher suits)
 - Negociación de cipher suites
 - Autenticacion
 - Intercambio/derivación de llaves
 - ChangeCipherspec
 - Finish (comprobación de hash)
 - Handshake TLS
 - Establecer identidad
 - Establecer llaves de sesion
 - * DH
 - * Transporte de llaves RSA
 - Transferencia en masa (bulk):
 - Se transfieren los datos de interes (payload)
 - Esa transferencia se hace con metodos simetricos.
 - Http
 - Es un protocolo peticion-respuesta
 - La cantidad de interacciones peticion-respuesta hay una sesion http es de una por sesion.
 - MiM en TLS
 - Cliente manda su hello que incluye la lista de cipher suite soportados

- Un tercero altera la lista de cipher suits dejando solo las debiles
- La forma de mitigacion es verificar el finished del handshake el hash de todo lo que se ha transmitido.
- Verificar certificados
- Verificar identidad -> reto-respuesta
- La criptografia es segura siempre y cuando no se comprometan los secretos
- Tiempo promedio para detectar una brecha es de 6 meses a 1 año.

3. Ventajas de TLS 1.3

- No tiene ciphersuits debiles
 - Nada con RSA -> transporte RSA
 - Solo AEAD
- Menor latencia
- Heartbeats (latido): En un sistema distribuido no puedes saber si un nodo esta activo o inactivo, solo puedes comprobarlo haciendo una petición, el heartbeat es una forma de comprobar si un nodo sigue activo cada x tiempo.

4.10.3 Verificación de certificados

- No debe de estar expirado
- No debe de estar revocado
- Se debe respetar la jerarquia de issuer siendo el subject del siguiente certificado en la cadena hasta llegar a la raiz.

4.10.4 Ataques a TLS

- Padding Oracle On Downgrade Legacy Encryption: TLS 1.0 puede ser explotado cuando se usa CBC. En caso de usarse 1.1 o 1.2, se puede re-negociar para usar el protocolo 1.0.
 - Deshabilitar TLS 1.0 y 1.1
 - Verificar TLS 1.2 mediante una herramienta de auditoria
 - Migrar a TLS 1.3

- Se forzo al servidor a bajar de versión para tener cipher suites debiles.
- Logjam: El hello-client es interceptado y cambia la lista del cipher suite a sus versiones EXPORT de DH.
- EXPORT: Son algoritmos debiles en la cipher suite dado que eran exportadas a otros gobiernos.
- FREAK: Ataca al transporte de llaves RSA usando versiones export.
 - Deshabilitar Cipher-suites debiles, en especial versiones EXPORT
 - Usa clientes que no acepten parametros debiles en DH/ECDH o RSA.
- Sweet32: Para cifradores de bloques de 64 bits como 3DES. Impacta en los datos que se pueden cifrar con la misma llave antes de repetirse en diferentes bloques. Dado que el limite es 32 Gb.
 - Deshabilitar cualquier cipher suite que use 3DES o cifradores de bloque de 64 bits.
- Return Of Bleichenbacher's Oracle Threat (ROBOT): Aprovecha las vulnerabilidades de padding en PKCS 1.5 para RSA.
 - Deshabilitar todos los cipher suites que usen RSA para intercambio de llaves.
- Existe un problema con la compresión que revela información a los atacantes, esto en la versión 1.2.
- Compression Ratio Info-Leak Made Easy (CRIME): La compresión sólo funciona si hay datos repetidos, si tu sólo tienes un texto cifrado que contiene un texto comprimido y si puedes insertar o parcialmente insertar mensajes para ver el efecto de la compresión y notas que la compresión a mejorado con la inserción del mensaje, eso quiere decir que agregaste algo repetido en el mensaje original
- TIME: Es mas efectivo que CRIME.
- BREACH: Ataca tambien la compresión para revelar información.
 - Deshabilitar compresión.
- Heartbleed: Hace una petición de heartbeat excediendo el tamaño real, pudiendo leer la memoria y su contenido.
 - Mantener las bibliotecas y aplicaciones actualizaciones.

4.11 Examen

- MACS, Firmas digitales, Certificados
- DH y transporte de RSA (cifrado hibrido)
- Cifrado simetrico AEAD
- TLS (versiones, handshake)
- Tareas/Proyectos